
Technical Documentation

AI-powered task redistribution system using Gemini + MongoDB

Version 1.0 | May 14, 2026

Prepared for: Engineering Team & Future Contributors

Table of Contents

- 1. Project Overview**
- 2. Architecture & System Design**
 - 2.1 High-Level Architecture
 - 2.2 Data Flow
- 3. File Structure**
- 4. Module Reference**
 - 4.1 ai_engine.py — Orchestrator
 - 4.2 tools.py — Tool Layer
 - 4.3 feed_data.py — Seed Script
- 5. Database Schema**
 - 5.1 Collections Overview
 - 5.2 Field Definitions
- 6. Tool API Reference**
 - 6.1 get_team_workload
 - 6.2 get_overtime_data
 - 6.3 get_urgent_deadlines
 - 6.4 get_reassignable_tasks
 - 6.5 find_best_candidates
 - 6.6 save_redistribution_suggestion
- 7. Configuration & Environment**
- 8. Setup & Running**
- 9. Scoring & Business Logic**
 - 9.1 Effective Hours Calculation
 - 9.2 Workload Level Classification
 - 9.3 Burnout Score Formula
 - 9.4 Candidate Composite Score
- 10. Known Bug Fixes & Gap Resolutions**
- 11. Extension Guide**

1. Project Overview

The **Workload Balancing Engine** is an autonomous AI agent that monitors a software engineering team's workload in real time and proactively recommends task redistributions to prevent burnout and missed deadlines. It combines Google's Gemini LLM (via function-calling / tool-use) with a MongoDB backend to analyse employee capacity, burnout risk, and task characteristics, then stores actionable redistribution suggestions for managers to review.

Objectives

- Detect overloaded engineers automatically (utilisation > 110 %).
- Identify burnout risk based on overtime hours, burnout-signal days, and average daily hours.
- Find the best qualified and available candidate to absorb each task using a composite scoring model.
- Surface actionable, deduplicated redistribution suggestions with full audit trail.
- Operate continuously without human intervention via a simple agent loop.

Technology Stack

Component	Technology	Purpose
AI / LLM	Google Gemini 2.5 Flash Lite	Decision-making, tool orchestration, report generation
AI SDK	google-genai (Python)	Gemini API client with function-calling support
Database	MongoDB	Stores employees, tasks, timesheets, suggestions
ORM / Driver	PyMongo	Direct MongoDB access with retry logic
Data Seeding	Faker + random	Generates 40 employees, 250 tasks, 1 200 timesheets
Env Management	python-dotenv	Loads API keys and DB config from .env
Retry Logic	tenacity	Exponential-back-off retries on PyMongoError
Language	Python 3.10+	Core runtime

2. Architecture & System Design

2.1 High-Level Architecture

The system follows a single-agent, agentic loop pattern. The orchestrator (`ai_engine.py`) drives all decisions through iterative calls to Gemini; each iteration either invokes one or more tools or produces a final answer.

Layer	Component	Responsibility
Entry Point	<code>ai_engine.py :: run_workload_analysis()</code>	Bootstraps the agent loop; handles history, iteration cap, context pruning
LLM	Gemini 2.5 Flash Lite	Decides which tool to call, interprets results, writes final report
Tool Layer	<code>tools.py :: run_tool()</code>	Dispatches tool names to Python functions; wraps errors safely
Business Logic	6 tool functions in <code>tools.py</code>	Workload calc, burnout scoring, candidate ranking, suggestion dedup
Persistence	MongoDB (6 collections)	employees, tasks, timesheets, projects, <code>assignment_history</code> , <code>redistribution_suggestions</code>
Seed / Fixtures	<code>feed_data.py</code>	Generates realistic synthetic dataset for development & testing

2.2 Data Flow

- **Startup** — `ensure_indexes()` is called to guarantee MongoDB indexes exist before any queries.
- **Initial prompt** — A structured user prompt is placed into the conversation history, instructing Gemini to run all analysis steps.
- **Gemini invocation** — The model is called with the full history and TOOLS list. It returns `function_call` parts.
- **Tool execution** — `run_tool()` dispatches each function call; results are appended to history as `role='tool'` content.
- **Iteration** — Steps 3–4 repeat until Gemini returns no function calls (final answer) or `MAX_ITERATIONS` (20) is reached.
- **History pruning** — When history exceeds 60 items (`30 turns × 2`), the oldest turns beyond the first are dropped to prevent context-window overflow.
- **Output** — The final text is printed and returned; redistribution suggestions are persisted in MongoDB.

3. File Structure

```
Workload_Balancing_Engine/  
  ai_engine.py      – Main orchestrator; Gemini agent loop  
  tools.py         – All tool functions + MongoDB helpers + TOOLS schema  
  feed_data.py    – One-time seed script; populates MongoDB with synthetic  
data  
  .env             – Runtime secrets (not committed)  
  .env.example    – Template showing all required env variables  
  __pycache__/     – Python bytecode cache (auto-generated, safe to delete)
```

4. Module Reference

4.1 ai_engine.py — Orchestrator

This is the entry point of the engine. It initialises the Gemini client, builds the system prompt and conversation history, and runs the agentic loop until a final report is generated.

Key constants

Constant	Value	Purpose
MODEL_NAME	gemini-2.5-flash-lite	Gemini model used for all completions
MAX_ITERATIONS	20	Hard cap on agent loop iterations
MAX_HISTORY_TURNS	30	Max conversation turns before history pruning

Key functions

Function	Signature	Description
run_workload_analysis	() → str	Main entry point. Runs the full agent loop and returns the final report text.
_convert_tools_for_gemini	() → list[Tool]	Converts the TOOLS list from tools.py into Gemini FunctionDeclaration objects.
_safe_text	(response) → str	Safely extracts text from Gemini response parts, avoiding crashes when no text parts exist.
_trim_large_args	(tool_args, max_list=20) → dict	Truncates any list-valued argument to max_list items before logging, preventing log bloat.
_trim_large_result	(result, max_list=20) → dict	Same as above but applied to tool result dicts.

System Prompt

The system prompt defines the agent's role and mandatory output format. Key rules encoded in it:

- Never reassign blocked tasks or tasks due within 24 hours.
- Prefer candidates with has_capacity = true (utilisation < 70 %).
- Prefer skill_score > 50 for strong matches.
- Always include exact numeric values in the final report.
- Follow a strict FINAL SUMMARY FORMAT covering team snapshot, overloaded employees, burnout risks, redistributions, and overall summary.

4.2 tools.py — Tool Layer

Contains all business logic, MongoDB access helpers, and the TOOLS schema consumed by the Gemini function-calling API. Every database-touching function is wrapped with the @_mongo_retry

decorator (3 attempts, exponential back-off) to handle transient connectivity issues.

Module-level constants

Constant	Value	Meaning
PRIORITY_HOUR_WEIGHT	{low:1.0, medium:1.3, high:1.6, critical:2.0}	Multiplier applied to raw hours based on task priority
BLOCKED_HOUR_PENALTY	0.25	Blocked tasks count for only 25 % of their weighted hours
OVERLOAD_THRESHOLD	1.10	Utilisation ratio above which an employee is overloaded
RISK_THRESHOLD	0.90	Utilisation ratio above which an employee is at risk
CAPACITY_THRESHOLD	0.70	Utilisation ratio below which an employee has spare capacity
BURNOUT_OVERTIME_7D	8 hrs	7-day overtime threshold for burnout risk flag
BURNOUT_SIGNAL_DAYS	3 days	Minimum consecutive burnout-signal days to flag risk
BURNOUT_AVG_HOURS_DAY	9.5 hrs	Average daily hours above which burnout risk is flagged

Internal helpers

Helper	Description
<code>_get_db()</code>	Lazy singleton MongoClient; reuses connection across calls.
<code>_effective_hours(task)</code>	Computes priority- and complexity-weighted remaining hours for a task.
<code>_strip_datetimes(obj)</code>	Recursively converts datetime objects to ISO strings to prevent JSON serialisation errors.
<code>_compute_burnout_score(employee_id, seven_days_ago)</code>	Aggregates timesheet data to produce <code>burnout_score</code> , overtime totals, and <code>is_burnout_risk</code> flag.

4.3 feed_data.py — Seed Script

A standalone script that wipes and repopulates all six MongoDB collections with realistic synthetic data using the Faker library. Run once before the first engine execution or whenever a fresh dataset is needed.

Collection	Records generated	Notes
employees	40	Random skills (5–10 per person), teams, designations, IST timezone

Collection	Records generated	Notes
projects	10	10 named projects with random risk scores and deadlines
tasks	250	Random assignments across employees and projects
timesheets	1 200	30 days × 40 employees; burnout_signal correlated with overtime
assignment_history	300	Historical quality and deadline-met rates per employee
redistribution_suggestions	0	Empty at seed time; populated by ai_engine.py at runtime

5. Database Schema

Database name: **workload_balancing_ai** (configurable via DB_NAME env var). All collections are schema-less (MongoDB document model) but follow the structures below.

5.1 employees

Field	Type	Description
employee_id	String (unique)	e.g. EMP-1000 — primary key
name	String	Full name
email	String	Work email
team	String	backend frontend ml devops data-engineering
designation	String	Job title
experience_years	Integer	Years of experience (1–12)
skills	Array[String]	Subset of 31 defined skills
capacity_hours_per_week	Integer	Weekly capacity (default 40)
max_safe_hours_per_week	Integer	Upper safe limit (typically 45)
timezone	String	Asia/Kolkata for all seed data
availability_status	String	available busy partially_available
performance_score	Float	65–98; overall performance rating
burnout_risk_score	Float	0.1–0.9; stored (seed), recomputed at runtime

5.2 tasks

Field	Type	Description
task_id	String (unique)	e.g. TASK-10000
title	String	Short task title
description	String	Longer description
project_id	String	Reference to projects.project_id
assigned_to	String	Reference to employees.employee_id
priority	String	low medium high critical
status	String	todo in_progress blocked review completed

Field	Type	Description
estimated_hours	Integer	Original time estimate
actual_hours	Integer	Hours actually spent
complexity_score	Float	1.0–10.0 task complexity
required_skills	Array[String]	Skills needed to complete this task
deadline	DateTime	UTC deadline
completion_percentage	Integer	0–100
blocked	Boolean	True when task is blocked
ai_risk_score	Float	0.1–0.95; AI-assessed task risk

5.3 timesheets

Field	Type	Description
timesheet_id	String (UUID)	Unique record ID
employee_id	String	Reference to employees.employee_id
date	DateTime	Day of entry (UTC)
hours_worked	Float	Total hours that day
overtime_hours	Float	$\max(\text{hours_worked} - 8, 0)$
productivity_score	Float	60–100; inversely correlated with overtime
meetings_count	Integer	0–8 meetings
tasks_completed	Integer	0–5 tasks finished
focus_time_hours	Float	Deep-work hours
burnout_signal	Boolean	True when overtime > 1h and random < 0.6

5.4 redistribution_suggestions

Field	Type	Description
task_id	String	Task being reassigned
task_title	String	Human-readable task title
from_employee_id	String	Current overloaded assignee
to_employee_id	String	Proposed new assignee

Field	Type	Description
to_employee_name	String	Name of new assignee
reason	String	Full plain-English explanation
urgency_score	Float	0.0–1.0; how urgently this should be done
confidence_score	Float	0.0–1.0; AI confidence in the match
status	String	pending approved rejected
created_at	DateTime	UTC timestamp of suggestion creation
acted_on	Boolean	False until a manager acts

6. Tool API Reference

All tools are exposed to Gemini as function declarations and dispatched via `run_tool(name, args)`. Every tool catches `PyMongoError` and unexpected exceptions, returning a structured error dict rather than raising, so the agent can continue.

6.x get_team_workload

Signature: `() → dict`

Fetches and computes the current workload for every employee. Returns a sorted list (highest effective hours first) plus a summary count of overloaded, at-risk, normal, and underutilised employees.

Returns

Key	Type	Description
team_workload	Array	One record per employee with utilisation, effective hours, burnout data, task counts
summary	Object	Aggregate counts: total, critical, overloaded, at_risk, normal, underutilised, burnout_risks

6.x get_overtime_data

Signature: `() → dict`

Computes 7-day overtime and burnout statistics for all employees by aggregating timesheets. Returns records sorted by burnout_score descending.

Returns

Key	Type	Description
overtime_data	Array	Per-employee: total_overtime_7d, burnout_signal_days, avg_hours_per_day, avg_productivity, burnout_score, is_at_risk
at_risk_count	Integer	Total employees with is_at_risk = True

6.x get_urgent_deadlines

Signature: `(days_ahead=7) → dict`

Returns all non-completed tasks and projects whose deadline falls within the next N days.

Inputs

Parameter	Type	Description
days_ahead	Integer	Optional. Look-ahead window in days (default 7)

Returns

Key	Type	Description
urgent_tasks	Array	Tasks sorted by deadline ascending
urgent_projects	Array	Projects sorted by deadline ascending

6.x get_reassignable_tasks

Signature: (employee_id) → dict

Returns all tasks belonging to employee_id that are safe to reassign: status in (todo, in_progress), not blocked, and deadline > 24 hours from now. Sorted by priority ascending (lowest priority first).

Inputs

Parameter	Type	Description
employee_id	String	Required. The overloaded employee's ID

Returns

Key	Type	Description
employee_id	String	The queried employee
reassignable_tasks	Array	Safe tasks with required_skills, deadline, priority, etc.
total_count	Integer	Number of reassignable tasks found

6.x find_best_candidates

Signature: (required_skills, exclude_employee_id) → dict

Ranks all available (not overloaded, not excluded) employees by a composite score. Employees with zero skill overlap are automatically excluded. Returns top 5 candidates.

Inputs

Parameter	Type	Description
required_skills	Array[String]	Required. Skills the task needs
exclude_employee_id	String	Required. Current assignee to exclude

Returns

Key	Type	Description
candidates	Array	Up to 5 candidates with skill_score, utilization_pct, composite_score, burnout_risk, matched_skills

6.x save_redistribution_suggestion

Signature: (suggestion: dict) → dict

Persists a redistribution suggestion. Validates all required fields and score ranges. Deduplicates: if a pending suggestion already exists for the same task_id, returns the existing record without inserting.

Inputs

Parameter	Type	Description
task_id	String	Required
task_title	String	Required
from_employee_id	String	Required
to_employee_id	String	Required
to_employee_name	String	Required
reason	String	Required. Plain-English explanation
urgency_score	Float 0–1	Required
confidence_score	Float 0–1	Required

Returns

Key	Type	Description
saved	Boolean	True if inserted, False if duplicate or validation error
suggestion_id	String	MongoDB ObjectId of inserted or existing record

7. Configuration & Environment

All secrets and tunable settings are loaded from a `.env` file in the project root via `python-dotenv`. Never commit the real `.env` to version control.

Variable	Default	Required	Description
GEMINI_API_KEY	—	YES	Google Gemini API key. Engine will not start without this.
MONGODB_URI	mongodb://localhost:27017	No	MongoDB connection string. Supports Atlas URI.
DB_NAME	workload_balancing_ai	No	Target database name.
SCHEDULE_INTERVAL_HOURS	6	No	Not consumed by engine directly; placeholder for scheduled runs.
LOG_LEVEL	INFO	No	Python logging level (DEBUG INFO WARNING ERROR).

Security Note

- Add `.env` to `.gitignore` before committing.
- For production, prefer environment variables injected by the deployment platform rather than a `.env` file.
- Rotate `GEMINI_API_KEY` regularly and scope it to the minimum required permissions.

8. Setup & Running

Prerequisites

- Python 3.10 or later
- MongoDB 6.0+ running locally or a MongoDB Atlas cluster
- A valid Google Gemini API key (Gemini 2.5 Flash Lite access)

Installation

Step 1: Clone / unzip the project

```
unzip Workload_Balancing_Engine.zip && cd Workload_Balancing_Engine
```

Step 2: Create a virtual environment

```
python -m venv .venv && source .venv/bin/activate
```

Step 3: Install dependencies

```
pip install google-genai pymongo python-dotenv tenacity faker
```

Step 4: Configure environment

```
cp .env.example .env
# then edit .env and add your keys
```

Step 5: Seed the database

```
python feed_data.py
```

Step 6: Run the engine

```
python ai_engine.py
```

Expected Output

The engine prints iteration-by-iteration tool calls and results to stdout, followed by a formatted report block. A sample structure:

```
=====
GEMINI WORKLOAD BALANCING ENGINE STARTED
=====

[Iteration 1] Calling Gemini...
→ Tool: get_team_workload
  Args: {}
  Result preview (0.123s): {'team_workload': [...], 'summary': {...}}
...
=====
FINAL GEMINI ANALYSIS
=====
=== WORKLOAD BALANCING REPORT ===
...
```

9. Scoring & Business Logic

9.1 Effective Hours Calculation

Raw estimated hours are adjusted for priority, complexity, and completion percentage before being used in utilisation calculations. Blocked tasks count for only 25 % of their weighted hours.

```
remaining = estimated_hours × (1 - completion_pct / 100)
priority_w = {low:1.0, medium:1.3, high:1.6, critical:2.0}[priority]
complexity_w = 0.8 + (complexity_score - 1) × (0.6 / 9)

if blocked:
    effective_hours = remaining × 0.25
else:
    effective_hours = remaining × priority_w × complexity_w
```

9.2 Workload Level Classification

Level	Condition	Action
critical	utilisation ≥ 1.10 AND critical_tasks ≥ 2	Highest priority for redistribution
overloaded	utilisation ≥ 1.10	Redistribution recommended
at_risk	0.90 ≤ utilisation < 1.10	Monitor; may escalate
normal	0.70 ≤ utilisation < 0.90	No action needed
underutilised	utilisation < 0.70	Preferred candidates for new tasks

9.3 Burnout Score Formula

The burnout score is a weighted composite of four signals, each normalised to 0–1:

```
overtime_s = min(total_overtime_7d / 8, 1.0)
signal_s = min(burnout_signal_days / 3, 1.0)
hours_s = min(max(avg_hrs_day - 8, 0) / 1.5, 1.0)
prod_inv = max(0, (80 - avg_productivity) / 80)

burnout_score = overtime_s×0.35 + signal_s×0.30 + hours_s×0.20 + prod_inv×0.15
```

An employee is flagged **is_burnout_risk = True** if any of the following holds: total_overtime_7d ≥ 8 h, burnout_signal_days ≥ 3, or avg_hours_per_day ≥ 9.5.

9.4 Candidate Composite Score

Candidates are ranked by a weighted composite of four dimensions:

Dimension	Weight	Calculation
Skill score	40 %	matched_skills / required_skills × 100

Dimension	Weight	Calculation
Capacity score	30 %	$\max(0, 100 - \text{utilisation_pct})$
Quality score	20 %	Average <code>quality_score</code> from <code>assignment_history</code> (default 70)
Deadline rate	10 %	$\text{deadlines_met} / \text{total_assignments} \times 100$ (default 70)

Candidates with zero skill overlap are excluded entirely before scoring.

10. Known Bug Fixes & Gap Resolutions

The codebase documents all intentional fixes with inline comments. The table below maps each fix to its impact.

ID	File	Description	Impact
BUG-1	tools.py	MongoClient uses connection pooling + timeouts	Prevents connection leaks under load
BUG-2	ai_engine.py	ensure_indexes() called before first query	Prevents missing-index query failures
BUG-3	tools.py	_strip_datetimes() recursively converts datetime → ISO string	Fixes JSON serialisation crash in tool responses
BUG-4	tools.py	Single aggregation pipeline for assignment history (all candidates at once)	Eliminates N+1 DB queries in find_best_candidates
BUG-5	ai_engine.py	History pruned when it exceeds MAX_HISTORY_TURNS × 2	Prevents Gemini context-window overflow on long runs
BUG-6	tools.py	capacity_hours_per_week defaults to 40 if None	Prevents ZeroDivisionError in utilisation calc
BUG-7	tools.py	find_best_candidates accepts optional workload snapshot	Avoids redundant DB scan when snapshot is available
BUG-8	tools.py	save_redistribution_suggestion validates required fields + score ranges	Prevents malformed documents in MongoDB
GAP-1	tools.py	Candidates with skill_score == 0 are skipped early	Avoids irrelevant suggestions
GAP-2	tools.py	Deduplication check before inserting a suggestion	Prevents duplicate pending suggestions per task
GAP-3	tools.py	ensure_indexes() creates all necessary MongoDB indexes	Query performance and uniqueness guarantees
GAP-4	tools.py	run_tool() catches PyMongoError and generic Exception	Agent continues safely if a single tool fails
GAP-5	tools.py	@_mongo_retry decorator with tenacity exponential back-off	Handles transient MongoDB connectivity issues

11. Extension Guide

This section provides practical guidance for future developers extending the engine.

Adding a New Tool

- 1. Implement the Python function in **tools.py** with the **@_mongo_retry** decorator.
- 2. Add a tool definition dict (name, description, input_schema) to the **TOOLS** list.
- 3. Add a dispatch branch in **run_tool()**.
- 4. Mention the new tool's purpose in the system prompt inside **ai_engine.py** if Gemini should call it autonomously.

Changing the AI Model

Update the **MODEL_NAME** constant in **ai_engine.py**. Ensure the new model supports function calling via the google-genai SDK. Gemini 2.5 Pro can be used for higher accuracy at greater cost.

Connecting a Real HR / Project Management System

- Replace MongoDB queries in tool functions with API calls to Jira, Linear, Asana, or your internal tooling.
- Retain the function signatures and return schemas — the agent does not care where data comes from.
- Add authentication tokens to **.env** and load them in the tool functions.

Scheduling Periodic Runs

The engine is designed to run on-demand. To schedule it, wrap **run_workload_analysis()** in a scheduler such as:

- **APScheduler** — lightweight, runs inside the same process.
- **Celery + Redis** — distributed task queue; suitable for production.
- **GitHub Actions / cron job** — simple external trigger; **SCHEDULE_INTERVAL_HOURS** in **.env** documents the intended cadence.

Adding a Web / API Layer

Wrap the engine in a FastAPI application to expose:

- **POST /analyse** — triggers a full workload analysis run.
- **GET /suggestions** — returns pending redistribution_suggestions from MongoDB.
- **PATCH /suggestions/{id}** — allows a manager to approve or reject a suggestion.

Extending the Skill Set

The **ALL_SKILLS** list in `feed_data.py` and the skills stored in the `employees` collection are free-text. To add new skills, append them to **ALL_SKILLS** and re-seed the database, or update employee documents directly.

End of Documentation | Workload Balancing Engine v1.0